

Aggregate Functions in SQL

1. Introduction

In relational databases, data is usually stored in **large tables containing many records**. Analyzing such data row by row is inefficient and impractical. To overcome this problem, SQL provides **Aggregate Functions**.

SQL Aggregate Functions are special functions that:

- Process **multiple rows**
- Perform **calculations on a column**
- Return **a single summarized value**

These functions help in:

- Finding totals and averages
- Identifying minimum and maximum values
- Counting records
- Analyzing patterns and trends in data

Aggregate functions are widely used in **reports, analytics, and decision-making queries**.

2. Key Characteristics of Aggregate Functions

Aggregate functions in SQL:

- Perform calculations such as **sum, average, minimum, maximum, and count**
- Ignore **NULL values** in most cases
- Return **one output value** per query or per group
- Are commonly used with:
 - GROUP BY
 - HAVING
 - ORDER BY

Important Note:

The only aggregate function that counts NULL values is COUNT(*).

3. Syntax of Aggregate Functions

AGGREGATE_FUNCTION(column_name)

Examples

SUM(Salary)

AVG(Marks)

COUNT(*)

4. Sample Table Used for Examples

EMPLOYEE Table

EmpID	Name	Department	Salary
1	Rahul	IT	45000
2	Anita	HR	38000
3	Mohan	IT	52000
4	Sita	Sales	30000
5	Kiran	HR	NULL

5. Aggregate Functions in SQL

5.1 COUNT() Function

Definition

The COUNT() function is used to **count the number of rows** in a table or the number of **non-NULL values** in a column.

Types of COUNT()

1. COUNT(*)

Counts **all rows**, including rows with NULL values.

```
SELECT COUNT(*) AS TotalRecords
```

```
FROM Employee;
```

Explanation:

- Returns the total number of rows in the table.

2. COUNT(column_name)

Counts only **non-NULL values** in the specified column.

```
SELECT COUNT(Salary) AS NonNullSalaries
```

```
FROM Employee;
```

Explanation:

- Rows with NULL salary are ignored.

3. COUNT(DISTINCT column_name)

Counts **unique non-NULL values**.

```
SELECT COUNT(DISTINCT Salary) AS UniqueSalaries
```

```
FROM Employee;
```

Explanation:

- Duplicate salary values are counted only once.

5.2 SUM() Function

Definition

The SUM() function calculates the **total of all non-NULL numeric values** in a column.

Example Queries

```
SELECT SUM(Salary) AS TotalSalary
```

```
FROM Employee;
```

Explanation:

- Adds all non-NULL salary values.

```
SELECT SUM(DISTINCT Salary) AS DistinctSalarySum
```

```
FROM Employee;
```

Explanation:

- Adds only **unique salary values**, avoiding duplicates.

Important Rules

- Works only with **numeric columns**
- Ignores NULL values

5.3 AVG() Function

Definition

The AVG() function calculates the **average (mean)** of all non-NULL numeric values in a column.

Example Queries

```
SELECT AVG(Salary) AS AverageSalary  
FROM Employee;
```

Explanation:

- Divides the sum of salaries by the number of non-NULL salary entries.

```
SELECT AVG(DISTINCT Salary) AS DistinctAvgSalary  
FROM Employee;
```

Explanation:

- Calculates the average using only unique salary values.

5.4 MIN() and MAX() Functions

Definition

- MIN() returns the **smallest value**
- MAX() returns the **largest value**

Example Queries

```
SELECT MAX(Salary) AS HighestSalary  
FROM Employee;  
SELECT MIN(Salary) AS LowestSalary  
FROM Employee;
```

Explanation

- NULL values are ignored
- Can be used with:
 - Numeric data
 - Character data (alphabetical order)
 - Date values

GROUP BY, HAVING, and ORDER BY Clauses in SQL

1. GROUP BY Clause

Introduction

In SQL, when we need to perform calculations **separately for different groups of data**, the GROUP BY clause is used.

It groups rows that have the **same values in specified columns** into summary rows.

Definition

The **GROUP BY clause** groups rows having identical values in one or more columns and allows aggregate functions to be applied **group-wise**.

Syntax

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)  
FROM table_name  
GROUP BY column_name;
```

Example

```
SELECT Department, COUNT(*) AS TotalEmployees
FROM Employee
GROUP BY Department;
```

Explanation

- Rows are grouped by Department
- COUNT(*) calculates employees in each department
- One result row is produced per department

Important Rules

- All non-aggregated columns in the SELECT clause **must appear in GROUP BY**
- GROUP BY is written **after WHERE** and **before HAVING**

2. HAVING Clause

Introduction

The WHERE clause cannot be used with aggregate functions.
To filter **grouped results**, SQL provides the **HAVING clause**.

Definition

The **HAVING clause** is used to filter groups created by GROUP BY based on **aggregate conditions**.

Syntax

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Example

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 40000;
```

Explanation

- Groups employees by department
- Calculates average salary per department
- Displays only departments with average salary > 40,000

WHERE vs HAVING

WHERE	HAVING
Filters rows	Filters groups
Used before grouping	Used after grouping
Cannot use aggregates	Uses aggregate functions

3. ORDER BY Clause

Introduction

After grouping and filtering data, it is often necessary to **display results in a specific order**. The ORDER BY clause is used for this purpose.

Definition

The **ORDER BY clause** sorts the result set in **ascending (ASC)** or **descending (DESC)** order.

Syntax

```
SELECT column_name  
FROM table_name  
ORDER BY column_name ASC | DESC;
```

Example

```
SELECT Department, SUM(Salary) AS TotalSalary  
FROM Employee  
GROUP BY Department  
ORDER BY TotalSalary DESC;
```

Explanation

- Calculates total salary per department
- Sorts departments by total salary in descending order

Default Sorting

- ASC (Ascending) is the default order

4. Order of Execution of Clauses

Although written in one order, SQL executes clauses as:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

Exercises for practice

Lab Exercise 1: Aggregate Functions with GROUP BY and HAVING

Aim To perform group-wise calculations using aggregate functions and SQL clauses.

Question

Write SQL commands in Oracle to perform the following operations:

1. Create a table **EMPLOYEE** with the following attributes:
 - EMP_ID – NUMBER (Primary Key)
 - EMP_NAME – VARCHAR2(30)
 - DEPARTMENT – VARCHAR2(20)
 - SALARY – NUMBER
2. Insert at least **6 records** into the EMPLOYEE table.
3. Display the **total salary department-wise**.
4. Display only those departments where the **total salary exceeds 80,000**.
5. Display the **average salary** of employees department-wise.
6. Sort the result in **descending order of total salary**.

Lab Exercise 2: COUNT and DISTINCT with GROUP BY

Aim To analyze records using COUNT and DISTINCT aggregate functions.

Question

Write SQL commands in Oracle to perform the following operations:

1. Create a table **STUDENT** with the following attributes:
 - STUD_ID – NUMBER (Primary Key)
 - STUD_NAME – VARCHAR2(30)
 - COURSE – VARCHAR2(30)
 - MARKS – NUMBER
2. Insert suitable records into the STUDENT table.
3. Display the **total number of students**.
4. Display the **number of students enrolled in each course**.
5. Display only those courses having **more than 2 students**.
6. Display the **count of distinct courses** offered.

Lab Exercise 3: SUM, AVG, MIN, and MAX Functions

Aim

To use multiple aggregate functions on numeric data.

Question

Write SQL commands in Oracle to perform the following operations:

1. Create a table **PRODUCT** with the following attributes:
 - PROD_ID – NUMBER (Primary Key)
 - PROD_NAME – VARCHAR2(30)
 - CATEGORY – VARCHAR2(20)
 - PRICE – NUMBER
2. Insert at least **5 records** into the PRODUCT table.
3. Display the **maximum and minimum price** of products.
4. Display the **average price category-wise**.
5. Display categories where the **average price is greater than 5,000**.
6. Display the **total price category-wise**, sorted in ascending order.

Lab Exercise 4: Aggregate Functions with WHERE Clause

Aim To combine WHERE clause with aggregate functions for filtered analysis.

Question

Write SQL commands in Oracle to perform the following operations:

1. Create a table **SALES** with the following attributes:
 - SALE_ID – NUMBER (Primary Key)
 - SALE_DATE – DATE
 - REGION – VARCHAR2(20)
 - AMOUNT – NUMBER
2. Insert suitable records into the SALES table.
3. Display the **total sales amount region-wise**.
4. Display the **total sales amount for the 'North' region only**.
5. Display regions where the **total sales exceed 1,00,000**.
6. Sort the results in **descending order of total sales**.

Lab Exercise 5: Comprehensive Aggregate Analysis

Aim To perform comprehensive data analysis using multiple SQL clauses and aggregate functions.

Question

Write SQL commands in Oracle to perform the following operations:

1. Create a table **ATTENDANCE** with the following attributes:
 - STUD_ID – NUMBER (Primary Key)
 - STUD_NAME – VARCHAR2(30)
 - CLASS – VARCHAR2(10)
 - ATTENDED_DAYS – NUMBER
2. Insert at least **6 records** into the ATTENDANCE table.
3. Display the **total attended days class-wise**.
4. Display the **average attendance per class**.
5. Display only those classes where the **average attendance is more than 75 days**.
6. Display the **number of students per class**, sorted in ascending order.